

# LegacyLift

An Agentic Legacy Modernization Accelerator with Test-Backed Safety,  
Powered by IBM Bob IDE

IBM Bob 2.0 Hackathon (lablab.ai) | 25-27 September 2026 | Team: [Team Name] | Members: [Member 1],  
[Member 2]

Repository: [github.com/GaziArnob/IBM-hackaton](https://github.com/GaziArnob/IBM-hackaton)

## Abstract

Modernizing legacy software is slow and risky: engineers must understand undocumented code and then change it without breaking behaviour nobody wrote down. LegacyLift is an agentic workflow built on IBM Bob IDE that turns this into a guided, test-backed pipeline: **Explain** the codebase, **Protect** it with characterization tests, **Migrate** it, and **Prove** behaviour was preserved. We applied it to the legacy Spring PetClinic application (Java 8, Spring Boot 2.7.3) and migrated it to **Java 21 and Spring Boot 3.4.5**. Bob added 30 characterization tests (41 to 71), raising branch coverage of the critical Owner and PetValidator classes to 100%. After migration all tests passed with **0 failures**, and test logic was verified to be unchanged. The full workflow took **1.7 hours** against an estimated 24 hours of manual work (**92.9% time saved**), using 39.23 Bobcoins across two team members.

## 1. Introduction

**1.1 Problem.** Many organizations still run business-critical systems on outdated frameworks and language versions. These systems are hard to change because the original authors have left, documentation is missing or stale, and automated tests are thin. Modernization projects are therefore frequently delayed, over budget or abandoned. The developer workflow that suffers most is *maintenance and migration*: time, effort and error rates are all high.

**1.2 Why an agent.** Migration needs more than autocomplete. It needs an agent that can read a whole repository, plan multi-step changes, generate and run tests, and explain its reasoning. IBM Bob IDE's agent mode provides this.

### 1.3 Contributions.

- **Explain:** architecture map, dependency diagram, migration inventory and test-gap analysis for an unfamiliar legacy repository.
- **Protect:** characterization tests written *before* any change, targeted at the gaps found in the analysis.
- **Migrate:** Java 8 / Spring Boot 2.7.3 to Java 21 / Spring Boot 3.4.5 under a strict rule that tests may not be changed.
- **Prove:** full re-run of all tests, a migration report, metrics and a dashboard.
- **Measured results** from JaCoCo, Surefire and git history, reported honestly, including where the agent was wrong.

## 2. Related Work

*Note to team: this section has reserved space. The short entries below are a starting point; expand each with proper citations and a clear statement of how LegacyLift differs.*

## 2.1 Rule-based code migration

Tools such as OpenRewrite apply deterministic refactoring recipes (for example, javax to jakarta or Spring Boot upgrades) across a codebase. They are reliable for mechanical changes but cannot explain code, reason about business logic, or handle cases that no recipe covers. **Gap:** no understanding or behaviour verification.

*[Space reserved: add 2-3 more rule-based or compiler-based migration approaches and compare.]*

## 2.2 AI-assisted code modernization

Industry assistants such as IBM watsonx Code Assistant for Z (COBOL to Java), Amazon Q Developer code transformation (Java version upgrades) and GitHub Copilot provide AI support for modernization. Most focus on code generation or a single transformation type. **Gap:** limited emphasis on generating behaviour-locking tests before migration and reporting evidence afterwards.

*[Space reserved: add academic work on LLM-based code translation and migration, with results and limitations.]*

## 2.3 Automated test generation

Search-based tools such as EvoSuite and random testing tools such as Randoop generate unit tests for Java automatically. More recent work uses large language models to generate readable tests. **Gap:** these tools are not usually integrated into an end-to-end migration workflow.

*[Space reserved: add LLM-based test generation studies and coverage results.]*

## 2.4 Characterization testing and agentic software engineering

Feathers (2004) introduced *characterization tests*: tests that record what legacy code currently does, so it can be changed safely. Benchmarks such as SWE-bench evaluate LLM agents on real repository-level tasks. LegacyLift combines both ideas: an agent that writes characterization tests first and then performs the migration.

*[Space reserved: add agent frameworks, multi-agent coding systems and relevant benchmarks.]*

## 2.5 Summary of positioning

| Approach                            | Explains code | Generates tests | Migrates   | Proves behaviour |
|-------------------------------------|---------------|-----------------|------------|------------------|
| Rule-based (OpenRewrite)            | No            | No              | Yes        | No               |
| AI assistants (general)             | Partly        | Partly          | Yes        | Rarely           |
| Test generators (EvoSuite, Randoop) | No            | Yes             | No         | Partly           |
| <b>LegacyLift (ours)</b>            | <b>Yes</b>    | <b>Yes</b>      | <b>Yes</b> | <b>Yes</b>       |

*[Space reserved: team may add more rows or a paragraph on novelty.]*

## 3. System Design

### 3.1 Workflow

LegacyLift runs as a four-stage pipeline. Each stage is driven by a detailed IBM Bob prompt in agent mode; the exact prompts are versioned in `PROMPTS.md` so the workflow can be repeated on another repository.

| Stage      | What Bob did                                                                                                                                                                                                                         | Output in the repository                                                                           |
|------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| 1. Explain | Read the repository, mapped packages and dependencies, traced one request end to end, listed every item blocking Java 21 / Spring Boot 3 and ranked the risks. Also compared the existing tests with the code to find coverage gaps. | reports/ARCHITECTURE.md (module map, Mermaid diagram, migration inventory, risks, test gaps)       |
| 2. Protect | Measured baseline coverage with JaCoCo, then wrote characterization tests only for the gaps found in stage 1, without touching production code.                                                                                      | 30 new tests (OwnerTests, PetValidatorTests, VisitTests + 3 controller tests), reports/COVERAGE.md |
| 3. Migrate | On a separate branch: upgraded pom.xml and the Maven wrapper, replaced javax with jakarta in 14 files, fixed JAXB and build-info issues. Tests were not allowed to change.                                                           | migration branch, reports/CHANGELOG.md                                                             |
| 4. Prove   | Re-ran all 71 tests on Java 21 and recorded coverage; then wrote the final report and machine-readable metrics.                                                                                                                      | reports/MIGRATION_REPORT.md, reports/metrics.json                                                  |

### 3.2 Architecture

- **Agent layer (IBM Bob IDE):** agent mode performs repository analysis, test generation, migration and reporting. Each prompt sets strict rules (for example "never change a test assertion") so the agent's output stays verifiable.
- **Tooling layer:** JDK 11 and JDK 21, Maven wrapper, JUnit 5, Mockito, AssertJ, MockMvc and JaCoCo. These give deterministic, measurable feedback on every change.
- **Reporting layer:** Markdown reports plus `metrics.json`, visualised by a Streamlit dashboard with KPI tiles, coverage and effort charts, the reports, and a Bob evidence tab.

*[Space reserved: insert architecture diagram here (export the Mermaid diagram from reports/ARCHITECTURE.md or draw.io).]*

## 4. Role of IBM Bob IDE

IBM Bob IDE is the core of LegacyLift. Every stage that needs understanding or judgement ran through Bob in agent mode, and two team members used their own Bob allocation:

| Member       | Bob tasks                                                                          | Bobcoins     |
|--------------|------------------------------------------------------------------------------------|--------------|
| Member 1     | Project setup, Explain, Protect, Migrate, Prove and migration report (Prompts 0-5) | 35.44        |
| Member 2     | Streamlit dashboard, dashboard fixes, documentation fact-check fixes (Prompts 6-8) | 3.79         |
| <b>Total</b> |                                                                                    | <b>39.23</b> |

Session consumption summaries for every task (task id, context length, Bobcoins) are stored as PNG screenshots in bob\_sessions/ and are also shown in the dashboard's "Bob Evidence" tab.

## 5. Tools and Technology Stack

| Category            | Tool (version used)                                      | Purpose                                                  |
|---------------------|----------------------------------------------------------|----------------------------------------------------------|
| AI agent (required) | IBM Bob IDE, agent mode                                  | Analysis, test generation, migration, reports, dashboard |
| Legacy runtime      | JDK 11, Java 8 target, Spring Boot 2.7.3                 | Build and test the original application                  |
| Target runtime      | Eclipse Temurin JDK 21, Spring Boot 3.4.5, Hibernate 6.6 | Migrated application                                     |
| Build               | Maven wrapper 3.8.2 → 3.9.9                              | Build, dependency management                             |
| Testing             | JUnit 5, Mockito, AssertJ, MockMvc                       | Characterization and regression tests                    |
| Coverage            | JaCoCo 0.8.7 → 0.8.12                                    | Baseline and post-migration coverage                     |
| Dashboard           | Python 3.12, Streamlit 1.45, pandas 2.2, Plotly 6.1      | Demo dashboard from metrics.json                         |
| Diagrams            | Mermaid                                                  | Dependency diagram in ARCHITECTURE.md                    |
| Collaboration       | Git + GitHub (main and migration branches)               | Version control, evidence trail                          |

## 6. Datasets and Compliance

The only dataset is public open-source code. No client data, company confidential data, personal information or social media content was used. Provenance is recorded in DATA\_SOURCES.md.

| Source                                                                                                               | Licence                           | Use                                   |
|----------------------------------------------------------------------------------------------------------------------|-----------------------------------|---------------------------------------|
| spring-projects/spring-petclinic, commit a5cbb85 (2022-10-16), the last commit targeting Java 8 with Spring Boot 2.x | Apache-2.0                        | Legacy application to modernize       |
| Maven Central dependencies                                                                                           | Per-artifact open-source licences | Resolved at build time, not committed |

## 7. Results

All numbers below are measured from JaCoCo, Surefire and git history, except the manual-effort figure, which is an estimate.

| Metric             | Before      | After             | How measured                   |
|--------------------|-------------|-------------------|--------------------------------|
| Java / Spring Boot | 1.8 / 2.7.3 | <b>21 / 3.4.5</b> | pom.xml, class file version 65 |

| Metric                                       | Before                 | After                    | How measured                        |
|----------------------------------------------|------------------------|--------------------------|-------------------------------------|
| Files with javax imports (excl. javax.cache) | 14                     | 0                        | grep                                |
| Tests                                        | 41 (original)          | 71 (+30 by Bob)          | Surefire reports                    |
| Failures after migration                     | -                      | 0 (70 passed, 1 skipped) | Surefire on JDK 21                  |
| Instruction coverage                         | 97.0%                  | 98.3% → <b>98.2%</b>     | JaCoCo: baseline → tests → migrated |
| Branch coverage                              | 88.7%                  | 96.8% → <b>96.8%</b>     | JaCoCo                              |
| Branch coverage: Owner / PetValidator        | 83.3% / 87.5%          | <b>100% / 100%</b>       | JaCoCo per class                    |
| Time for the full workflow                   | 24 h (manual estimate) | <b>1.7 h</b>             | git history 14:08 → 15:48           |
| Time saved                                   | -                      | <b>92.9%</b>             | (24 - 1.7) / 24                     |

**Tests as proof.** The migration prompt forbade any change to test assertions. We verified this independently: after removing imports and whitespace, the logic of every changed test file is identical before and after migration. The 0-failure result therefore shows preserved behaviour, not adjusted tests.

**Human verification caught agent mistakes.** Bob's first analysis listed 10 files with javax imports instead of 14 (missing JAXB in Vets.java) and suggested renaming javax.cache, which would have broken the build. Reviewing the analysis against the code and using grep as the source of truth in the migration prompt avoided both problems. The original analysis is kept, with a corrections note, in reports/ARCHITECTURE.md.

## 8. Impact, Limitations and Future Work

**Impact.** A developer gets a trustworthy path from unfamiliar legacy code to a modern stack, with documentation, tests and a report as by-products, and evidence that behaviour was preserved.

| Limitation                                                         | Next step                                                                         |
|--------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| Manual effort of 24 h is an estimate, not a controlled measurement | Time a developer doing one module manually and compare                            |
| 2 branches in PetController are still not covered (83.3%)          | Add a test for the petId == null path of findPet                                  |
| @MockBean is deprecated in Spring Boot 3.4 (4 test classes)        | Replace with @MockitoBean                                                         |
| The Gradle build was not migrated; only Maven                      | Migrate build.gradle or remove it                                                 |
| Evaluated on one Java repository                                   | Run the same prompts on more legacy repositories and other languages (e.g. COBOL) |

**Conclusion.** LegacyLift shows how IBM Bob IDE can go beyond autocomplete into a complete, evidence-backed modernization workflow: explain, protect, migrate and prove, in under two hours for a real Spring application, with zero regressions.

## 9. Submission Checklist

- GitHub repository with source code, README and setup steps: done

- bob\_sessions/ with a summary screenshot for every Bob task: done (7 files, 2 members)
- DATA\_SOURCES.md with repository, commit and licence: done
- Reports, metrics.json and Streamlit dashboard: done
- Demo video showing the four stages and the dashboard: [to do]

## References

- [1] M. Feathers, *Working Effectively with Legacy Code*. Prentice Hall, 2004.
- [2] OpenRewrite, automated source code refactoring. <https://docs.openrewrite.org>
- [3] G. Fraser and A. Arcuri, "EvoSuite: automatic test suite generation for object-oriented software," in *Proc. ESEC/FSE*, 2011.
- [4] C. Pacheco and M. D. Ernst, "Randoop: feedback-directed random testing for Java," in *OOPSLA Companion*, 2007.
- [5] C. E. Jimenez et al., "SWE-bench: Can language models resolve real-world GitHub issues?" in *Proc. ICLR*, 2024.
- [6] IBM, IBM Bob IDE Hackathon Guide, 2026.
- [7] [Team: add further references for Related Work here.]